

REACT EXAM — GUIDE DES ETAPES

Comment ajouter une page, un formulaire ou une route

1. AJOUTER UNE NOUVELLE PAGE

ETAPE 1: Creer le fichier du composant dans src/components/

```
src/components/MaPage.tsx
```

ETAPE 2: Ecrire le composant de base

```
import { useNavigate } from 'react-router-dom'

function MaPage() {
  const navigate = useNavigate()
  return (
    <div>
      <h2>Ma Page</h2>
      <button onClick={() => navigate('/')}>Retour</button>
    </div>
  )
}
export default MaPage
```

ETAPE 3: Ajouter la route dans App.tsx

```
import MaPage from './components/MaPage'

<Route path='/ma-page' element={ <MaPage /> } />
```

ETAPE 4: Ajouter un bouton pour naviguer vers cette page

```
<button onClick={() => navigate('/ma-page')}>Aller a MaPage</button>
```

2. AJOUTER UN FORMULAIRE

Option A: Simple (sans Zod)

ETAPE 1: Creer un useState par champ

```
const [name, setName] = useState('')
const [price, setPrice] = useState(0)
const [error, setError] = useState('')
```

ETAPE 2: Ecrire la fonction de validation manuelle

```
const handleSubmit = () => {
  if (!name) { setError('Nom obligatoire'); return }
  if (price < 0) { setError('Prix invalide'); return }
  addItem({ name, price })
  navigate('/')
}
```

ETAPE 3: Ecrire le JSX du formulaire

```
<input value={name} onChange={(e) => setName(e.target.value)} />
{error && <p style={{color:'red'}}>{error}</p>}
<button onClick={handleSubmit}>Ajouter</button>
```

Option B: Avec Zod + react-hook-form

ETAPE 1: Définir le schema Zod

```
const schema = z.object({
  name: z.string().min(1, 'Obligatoire'),
  price: z.number().min(0, 'Prix positif')
})
type MyForm = z.infer<typeof schema>
```

ETAPE 2: Initialiser useForm avec zodResolver

```
const { register, handleSubmit, reset, formState: { errors } } =
  useForm<MyForm>({ resolver: zodResolver(schema) })
```

ETAPE 3: Ecrire onSubmit

```
const onSubmit = (data: MyForm) => {
  addItem(data)
  reset()
  navigate('/')
}
```

ETAPE 4: Ecrire le JSX

```
<form onSubmit={handleSubmit(onSubmit)}>
  <input {...register('name')} />
  {errors.name && <p style={{color:'red'}}>{errors.name.message}</p>}
  <input type='number' {...register('price',{valueAsNumber:true})} />
  {errors.price && <p style={{color:'red'}}>{errors.price.message}</p>}
  <button type='submit'>Ajouter</button>
</form>
```

3. AJOUTER UN FORMULAIRE DE MODIFICATION (UPDATE)

La difference avec AddForm: pre-remplir les champs avec les donnees existantes.

ETAPE 1: Ajouter la route avec :id dans App.tsx

```
<Route path='/update/:id' element={<UpdateItem />} />
```

ETAPE 2: Ajouter le bouton Modifier dans la liste

```
<button onClick={() => navigate(`/update/${item.id}`)}>Modifier</button>
```

ETAPE 3: Dans UpdateItem.tsx — recuperer l'id et l'item

```
const { id } = useParams()
const items = useItemStore((state: any) => state.items)
const fetchItems = useItemStore((state: any) => state.fetchItems)
const item = items.find((i: any) => i.id === id) // id est string!
```

ETAPE 4: Fetch si items vide + pre-remplir le formulaire

```
useEffect(() => {
  if (items.length === 0) fetchItems()
}, [])

// AVEC ZOD: utiliser reset()
useEffect(() => {
  if (item) reset({ name: item.name, price: item.price })
}, [item])
```

```
// SANS ZOD: utiliser setName(), setPrice()
useEffect(() => {
  if (item) { setName(item.name); setPrice(item.price) }
}, [item])
```

ETAPE 5: Soumettre la modification

```
const onSubmit = (data: MyForm) => {
  updateItem(id, { ...item, ...data })
  navigate('/')
}
```

NOTE: IMPORTANT: Ne pas utiliser Number(id) — les ids json-server sont des strings!

4. ROUTES AVEC PARAMETRES

Type de route	App.tsx	Comment recuperer
Route simple	<code><Route path='/list' ... /></code>	rien a recuperer
Route avec id	<code><Route path='/details/:id' ... /></code>	<code>const { id } = useParams()</code>
Route avec donnees	<code><Route path='/reserve' ... /></code>	<code>const loc = useLocation(); loc.state?.item</code>

ETAPE 1: Route simple

```
<Route path='/list' element={<ItemList />} />
navigate('/list')
```

ETAPE 2: Route avec id (useParams)

```
<Route path='/details/:id' element={<Details />} />
navigate(`/details/${item.id}`)
// Dans Details.tsx:
const { id } = useParams()
```

ETAPE 3: Passer des donnees entre pages (useLocation)

```
// Page 1 — envoyer
navigate('/reserve', { state: { item: selectedItem } })

// Page 2 — recevoir
const location = useLocation()
const item = location.state?.item
```

5. LAZY LOADING

Lazy loading = charger les composants SEULEMENT quand on en a besoin (pas tout au debut).
Avantage: l'app démarre plus vite car elle ne charge pas tout d'un coup.

ETAPE 1: Remplacer les imports normaux par lazy()

```
// AVANT (import normal — charge tout au debut):
import ItemList from './components/ItemList'
import AddItem from './components/AddItem'
```

```
// APRES (lazy loading - charge seulement quand besoin):
import { lazy, Suspense } from 'react'

const ItemList = lazy(() => import('./components/ItemList'))
const AddItem = lazy(() => import('./components/AddItem'))
const UpdateItem = lazy(() => import('./components/UpdateItem'))
```

ETAPE 2: Entourer les Routes avec Suspense

```
export default function App() {
  return (
    <div>
      <Navbar />
      <Suspense fallback={<p>Chargement...</p>}>
        <Routes>
          <Route path="/" element={<ItemList />} />
          <Route path="/add" element={<AddItem />} />
          <Route path="/update/:id" element={<UpdateItem />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </Suspense>
    </div>
  )
}
```

NOTE: *fallback* = ce qui s'affiche pendant le chargement. Peut être un spinner ou juste du texte.

Resumé lazy loading en 2 regles:

1. Remplacer `import X from '...'` par `const X = lazy(() => import('...'))`
2. Entourer `<Routes>` avec `<Suspense fallback={<p>Chargement...</p>}>`

	Import normal	Lazy loading
Quand charge?	Au démarrage de l'app	Quand on visite la page
Vitesse démarrage	Plus lente	Plus rapide
Code	<code>import X from '...'</code>	<code>lazy(() => import('...'))</code>
Suspense needed?	Non	Oui obligatoire

6. AJOUTER UNE ACTION AU STORE ZUSTAND

ETAPE 1: Ajouter l'action dans le store

```
const useItemStore = create((set) => ({
  items: [],

  // action existante...
  fetchItems: async () => { ... },

  // NOUVELLE ACTION:
  incrementVues: async (item: any) => {
    const updated = { ...item, nbVues: item.nbVues + 1 }
    const res = await update(item.id, updated)
    set((state: any) => ({
      items: state.items.map((i: any) =>
        i.id === item.id ? res.data : i
      )
    }))
  })
}))
```

```
}  
}))
```

ETAPE 2: Utiliser dans le composant

```
const incrementVues = useItemStore((state: any) => state.incrementVues)  
<button onClick={() => incrementVues(item)}>+1 Vue</button>
```

7. CHECKLIST EXAMEN — ORDRE A SUIVRE

Au debut de l'examen, faire dans cet ordre:

Ordre	Action	Fichier
1	Creer le projet Vite	Terminal
2	Installer les packages	Terminal
3	Configurer main.tsx (BrowserRouter)	src/main.tsx
4	Creer db.json avec les donnees	src/api/db.json
5	Creer api.ts (axios)	src/api/api.ts
6	Creer le store Zustand	src/store/useItemStore.ts
7	Creer les composants (List, Add, Update...)	src/components/
8	Configurer App.tsx (Routes)	src/App.tsx
9	Tester chaque fonctionnalite	Navigateur

Commandes de demarrage:

```
npm create vite@latest nom_prenom -- --template react-ts  
cd nom_prenom  
npm install react-router-dom axios react-hook-form @hookform/resolvers zod zustand  
npm run dev # Terminal 1  
npx json-server --watch src/api/db.json --port 3001 # Terminal 2
```

Seul le code executable compte — bonne chance!